2. Categorical and Continuous Variables

To ensure the analysis aligns with the research objectives, specific variables have been included or excluded based on their relevance to customer classification and segmentation within the insurance industry. The selection process is guided by the need to capture essential demographic, financial, and behavioural characteristics that influence insurance purchasing patterns.

Included Variables

The categorical and continuous variables retained in the dataset play a significant role in defining customer profiles. They enable the classification of customers into predefined categories and allow unsupervised segmentation to reveal natural groupings.

Categorical Variables

The `Customer_Type` variable is essential as it is the target variable for classification tasks, enabling the model to predict customer subtypes based on other characteristics. `Avg_Age` is included to facilitate grouping by age demographics, categorising customers into segments such as **Young Adults**, **Middle-Aged Adults**, and **Seniors**, which may exhibit distinct insurance behaviours. `"Household_Profile"` is selected as it defines household structures, such as **Singles** and **Families with Children**, which influence financial stability and policy needs. `"Mobile_Home_Policies"` is retained to capture information regarding housing conditions, an essential factor in determining a customer's economic status and insurance preferences. The inclusion of `"Private_Third_Party_Insurance_Contribution"`, `"Business_Third_Party_Insurance_Contribution"`, and `"Agricultural_Third_Party_Insurance_Contribution"` is justified as these variables serve as indicators of economic stability and occupational risks, helping segment customers based on insurance needs.

Continuous Variables

Several demographic variables are retained to account for household composition and financial stability. `"Number_of_Houses"` and `"Avg_Household_Size"` contribute to the assessment of housing conditions, influencing the demand for insurance coverage. Variables such as `"Household_Without_Children"`, `"Household_With_Children"`, `"Married"`, `"Living_Together"`, `"Singles"`, and `"Other_Relation"` are relevant for understanding the structure of households, which impacts financial priorities and insurance engagement.

Educational and occupational variables such as `"High_Education_Level"`, `"Medium_Education_Level"`, `"Low_Education_Level"`, `"High_Status"`, `"Entrepreneur"`, `"Farmer"`, `"Middle_Management"`, `"Skilled_Labourers"`, and `"Unskilled_Labourers"` are retained as they strongly influence economic stability and purchasing behavior. Higher education levels typically correlate with increased financial literacy and insurance adoption, whereas certain occupations may indicate a greater need for specific types of insurance.

Social class indicators, including `"Social_Class_A"`, `"Social_Class_B1"`, `"Social_Class_B2"`, `"Social_Class_C"`, and `"Social_Class_D"`, are critical in differentiating spending behaviours and insurance preferences among economic groups. Customers belonging to higher social classes may exhibit greater financial security and higher insurance engagement.

Financial indicators such as `"Income_Less_Than_30K"`, `"Income_30K_to_45K"`, `"Income_45K_to_75K"`, `"Income_75K_to_122K"`, `"Income_Above_123K"`, `"Average_Income"`, and `"Purchasing_Power_Class"` are key variables that influence the affordability and willingness to purchase insurance policies.

Insurance contributions and policy metrics are essential for assessing customer behaviour in the insurance market. Variables such as `"Life_Insurance_Contribution"`, `"Private_Accident_Insurance_Contribution"`, `"Family_Accident_Insurance_Contribution"`, `"Disability_Insurance_Contribution"`, `"Fire_Insurance_Contribution"`, and `"Property_Insurance_Contribution"` provide insight into how customers allocate resources towards financial protection. Additionally, vehicle and property ownership indicators such as `"Rented_House"`, `"Home_Owner"`, `"Owns_One_Car"`, `"Owns_Two_Cars"`, and `"Owns_No_Car"` contribute to segmentation by identifying levels of financial security and asset accumulation.

Excluded Variables

Certain variables have been excluded from the dataset due to their limited influence on customer classification and segmentation.

Insurance types that cater to niche markets, such as `Surfboard_Insurance_Contribution` and `Boat_Insurance_Contribution`, have been omitted because they do not provide substantial differentiation between customer segments. Similarly, `"Mobile_Home_Policies_Count"` has been excluded, as `"Mobile_Home_Policies"` already captures relevant housing-related information.

General insurance access indicators, such as `"National_Health_Insurance"` and `"Social_Security_Insurance"`, are excluded because they are not strong predictors of private insurance preferences. These variables may have value in welfare analysis, but are not key determinants of customer profiling.

Industry-specific insurance types, such as `"Agricultural_Third_Party_Insurance_Contribution"`, `"Trailer_Policy_Contribution"`, `"Tractor_Policy_Contribution"`, and `"Moped_Policy_Contribution"`, have been omitted, as they are not broadly relevant across all customer groups. These variables may have significance within specific occupations but do not contribute meaningfully to general customer segmentation.

Conclusion

The dataset has been optimized by selecting variables directly linked to **income**, **education**, **social class**, **insurance contributions**, and **ownership**, ensuring a strong foundation for customer profiling and predictive analytics. Removing irrelevant variables enhances the **focus** of the analysis while preserving the model's ability to generate meaningful insights into customer behaviour. This selection process supports both **supervised classification tasks** and **unsupervised clustering techniques**, facilitating a deeper understanding of how customer attributes drive insurance-related decisions.

In [2]:
```
# PACKAGES ----
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.gridspec import GridSpec
from matplotlib.ticker import FuncFormatter
import seaborn as sns

# Suppress warnings
import warnings # reduce cluttered output
warnings.filterwarnings("ignore")

from google.colab import drive
drive.mount('/content/drive')

# Load data
# ins = insurance [shorthand dataframe name]
ins = pd.read_csv("/content/drive/MyDrive/Colab Notebooks/data_mining_2/insurance_2.csv")

# Filtered Categorical and Continuous variables that align with aims and objectives
CATEGORICAL_COLUMNS = [
    "customer_type",  # Target variable
    "avg_age",  # Age-based groupings (Young Adults, Seniors, etc.)
    "household_classification",  # Defines household structure affecting insurance needs
    "mobile_home_policy",  # Indicates housing stability & economic class
    "private_third_party",  # Financial indicator
    "business_third_party",  # Entrepreneurship insight
    "agricultural_third_party"  # Rural demographic indicator
]

CONTINUOUS_COLUMNS = ["total_properties_owned", "avg_household_size", "childless_households", "households_with_children",
                      "married", "living_together", "extended_family_members", "singles",
                      "highly_educated", "moderately_educated", "low_education",
                      "high_status_professionals", "entrepreneurs", "farmer", "middle_management",
                      "skilled_workers", "unskilled_workers",
                      "social_class_a", "social_class_b1", "social_class_b2", "social_class_c", "social_class_d",
                      "rented_house", "home_owner",
                      "single_car", "dual_car", "no_car",
                      "income_below_30k", "income_30k_to_45k", "income_45k_to_75k", "income_75k_to_122k", "income_above_123k",
                      "mean_income", "purchasing_power_group",
                      "car_insurance", "van_insurance", "bike_insurance", "lorry_insurance", "trailer_insurance", "tractor_insurance",
                      "agrimachine_insurance", "moped_insurance", "life_insurance", "private_accident_insurance", "family_accident_insurance",
                      "disability_insurance", "fire_insurance", "property_insurance", "social_security_insurance",
                      "private_third_party_policies_count", "business_third_party_policies_count",
                      "agricultural_third_party_policies_count", "car_policy_count", "van_policy_count",
                      "bike_policy_count", "lorry_policy_count", "trailer_policy_count", "tractor_policy_count",
                      "agrimachine_policy_count", "moped_policy_count", "life_insurance_count",
                      "personal_accident_count", "family_accident_count", "disability_insurance_count",
                      "fire_insurance_count", "property_insurance_count", "social_security_count",
                      "mobile_home_policy_count"
                     ]

print('Categorical columns')
print(CATEGORICAL_COLUMNS)
print(' ')
print('Continuous columns')
print(CONTINUOUS_COLUMNS)
```

```
Mounted at /content/drive
Categorical columns
['customer_type', 'avg_age', 'household_classification', 'mobile_home_policy', 'private_third_party', 'business_third_party', 'agricultural_third_party']

Continuous columns
['total_properties_owned', 'avg_household_size', 'childless_households', 'households_with_children', 'married', 'living_together', 'extended_family_members', 'singles', 'highly_educated', 'moderately_
educated', 'low_education', 'high_status_professionals', 'entrepreneurs', 'farmer', 'middle_management', 'skilled_workers', 'unskilled_workers', 'social_class_a', 'social_class_b1', 'social_class_b2',
'social_class_c', 'social_class_d', 'rented_house', 'home_owner', 'single_car', 'dual_car', 'no_car', 'income_below_30k', 'income_30k_to_45k', 'income_45k_to_75k', 'income_75k_to_122k', 'income_above_
123k', 'mean_income', 'purchasing_power_group', 'car_insurance', 'van_insurance', 'bike_insurance', 'lorry_insurance', 'trailer_insurance', 'tractor_insurance', 'agrimachine_insurance', 'moped_insuran
ce', 'life_insurance', 'private_accident_insurance', 'family_accident_insurance', 'disability_insurance', 'fire_insurance', 'property_insurance', 'social_security_insurance', 'private_third_party_poli
cies_count', 'business_third_party_policies_count', 'agricultural_third_party_policies_count', 'car_policy_count', 'van_policy_count', 'bike_policy_count', 'lorry_policy_count', 'trailer_policy_coun
t', 'tractor_policy_count', 'agrimachine_policy_count', 'moped_policy_count', 'life_insurance_count', 'personal_accident_count', 'family_accident_count', 'disability_insurance_count', 'fire_insurance_
count', 'property_insurance_count', 'social_security_count', 'mobile_home_policy_count']
```

---

3. Missing Data

[Disclaimer]: The code highlighted was supported by Microsoft Copilot, an AI-powered assistant to provide assistance in python code generation and debugging.

Effective management of missing data is essential to ensuring the reliability and validity of datasets for analysis. Missing values, if unaddressed, can lead to biased outcomes, reduced statistical power, and diminished representativeness. To mitigate these issues, tailored imputation strategies are applied based on data type. For categorical variables such as `mobile_home_policies_count`, `mobile_home_status`, and `business_third_party_policies`, gaps are replaced with the mode—the most frequently occurring value—to preserve the discrete patterns inherent in these data. Numerical variables such as `disability_insurance_premium`, `fire_insurance_premium`, and `life_insurance_premium` are imputed using the median to mitigate the impact of outliers while maintaining the dataset's statistical balance. By systematically addressing these gaps, the dataset retains its structural integrity and provides a robust foundation for unbiased analysis.

In [3]:
```python
# Check for missing data
missing_data = ins.isnull().sum()
missing_percentage = (missing_data / len(ins)) * 100

# Create a DataFrame to display the missing data information
missing_data_ins = pd.DataFrame({'Missing Values': missing_data, 'Percentage': missing_percentage})

# Format the 'Percentage' column to display as percentages with '%' sign [see disclaimer]
missing_data_ins['Percentage'] = missing_data_ins['Percentage'].apply(lambda x: f"{x:.1f}")

# Filter and sort the DataFrame
missing_data_ins = missing_data_ins[missing_data_ins['Missing Values'] > 0].sort_values(by='Percentage', ascending=False)

# Display the missing data information
missing_data_ins
```

Out[3]:

	Missing Values	Percentage
mobile_home_policy_count	74	1.3
disability_insurance	72	1.3
mobile_home_policy	65	1.2
life_insurance	60	1.1
fire_insurance	55	1.0
bicycle_insurance	52	0.9
social_security_count	49	0.9
property_insurance_count	49	0.9
agricultural_third_party_policies_count	52	0.9
business_third_party_policies_count	52	0.9
private_third_party_policies_count	52	0.9
property_insurance	52	0.9
social_security_insurance	52	0.9
boat_insurance	52	0.9
surfboard_insurance	52	0.9
family_accident_insurance	52	0.9
private_accident_insurance	52	0.9
moped_insurance	52	0.9
trailer_policy_count	32	0.6
tractor_policy_count	32	0.6
agrimachine_policy_count	32	0.6
moped_policy_count	32	0.6
life_insurance_count	32	0.6
personal_accident_count	32	0.6
family_accident_count	32	0.6
trailer_insurance	19	0.3
van_insurance	19	0.3
avg_age	14	0.3
private_third_party	9	0.2

Rationale for replacing Missing Data with Median or Mode

Median for Numerical Variables

The median is a robust measure of central tendency, particularly effective for numerical variables influenced by outliers. Continuous data such as premiums (e.g., `fire_insurance`) and contributions (e.g., `moped_insurance`) benefit from this as they provide an accurate representation of typical values without distortion from extreme observations.

Mode for Categorical Variables

The mode is a reliable measure for categorical variables, effectively preserving their discrete nature and observed patterns. This approach is particularly beneficial for attributes such as *policy counts* (e.g., `mobile_home_policy_count` and `trailer_policy_count`), where it ensures consistency across classifications and upholds the integrity of categorical data by addressing missing values without introducing distortion.

This hybrid approach, employing the median for numerical variables and the mode for categorical variables, adeptly addresses the challenges of diverse data types, strengthens the dataset's representativeness and reliability, and optimises the precision of subsequent analyses and modelling processes.

In [4]:
```python
# Replace missing values with mode for categorical variables and median for numerical variables
for column in ins.columns:
    if ins[column].dtype == 'object' or ins[column].dtype == 'category':  # Categorical variables
        mode_value = ins[column].mode()[0]  # Calculate the mode
        ins[column].fillna(mode_value, inplace=True)
    else:  # Numerical variables
        median_value = ins[column].median()  # Calculate the median
        ins[column].fillna(median_value, inplace=True)

# Check if missing values are replaced
missing_data2 = pd.DataFrame({
    'Missing Values': ins.isnull().sum().values,  # Total missing values for each column
    'Percentage': (ins.isnull().sum() / len(ins) * 100).values  # Percentage of missing values
})
```

```
print(missing_data2)
ins.to_csv("/content/drive/MyDrive/Colab Notebooks/data_mining_2/insurance_cleaned_missing.csv", index=False)
```

```
    Missing Values  Percentage
0                0         0.0
1                0         0.0
2                0         0.0
3                0         0.0
4                0         0.0
..             ...         ...
78               0         0.0
79               0         0.0
80               0         0.0
81               0         0.0
82               0         0.0

[83 rows x 2 columns]
```

---

4. Outliers

Outlier Detection and Handling with DBSCAN for Insurance Customer Classification

Introduction

This document details a comprehensive approach to outlier detection and handling using DBSCAN (Density-Based Spatial Clustering of Applications with Noise) in the context of insurance customer data analysis. The methodology presented supports the primary aims of this analysis:

1. **Classification**: Predicting and categorizing customers into relevant subtypes (e.g., "Rural & Low-Income", "Wealthy & Affluent")
2. **Segmentation**: Exploring natural groupings among customer profiles using unsupervised learning methods

Outliers can significantly impact both classification and segmentation tasks, potentially leading to biased models and inaccurate customer profiling. The robust handling of outliers is therefore crucial for developing reliable predictive models and meaningful customer segments that can guide business strategies within the insurance industry.

In [5]:
```
# PACKAGES ----
# Outlier Detection
from scipy import stats
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import DBSCAN
from sklearn.neighbors import NearestNeighbors
from collections import Counter

# Suppress warnings
#import warnings # reduce cluttered output
#warnings.filterwarnings("ignore")

#from google.colab import drive
#drive.mount('/content/drive')

#drive.mount("/content/drive", force_remount=True)
```

4.1 Detect outliers using the IQR method

4.1.1 Outliers Table

The following code implements a **systematic approach** for detecting and visualizing outliers in a dataset using the **interquartile range (IQR) method**.

- **Outlier Detection and Filtering**
  - Identifies extreme values that significantly deviate from the typical distribution.
  - Applies the IQR method to compute lower and upper bounds for each continuous variable.
  - Filters variables where more than **1% of data points** are detected as outliers, ensuring statistical relevance.
- **Data Visualization with an Accessible Design**
  - Utilizes a **color blindness-friendly palette** to enhance readability and accessibility.
  - Assigns **distinct colors** based on **socioeconomic categories**, ensuring clear differentiation.
  - Incorporates **aesthetic refinements**, such as large-font labels and a structured legend.
- **Optimized Presentation for Interpretation**
  - Removes **gridlines** to maintain a clean visual appearance.
  - Ensures **y-axis ticks are displayed**, improving reference points for data comparison.
  - Enhances **academic and professional readability**, making insights actionable in research settings.
- **Focused Exploration of Data Variability**
  - Helps researchers identify **potential anomalies** affecting socioeconomic trends.
  - Facilitates **data-driven decision-making** by pinpointing extreme data points.
  - Supports a **statistically sound approach** to handling outlier distributions in analysis.

This approach ensures that **meaningful outlier trends** are highlighted, fostering **a deeper understanding of variability** in socioeconomic data while maintaining clarity in visualization.

In [6]:
```
# Load data
ins2 = pd.read_csv("/content/drive/MyDrive/Colab Notebooks/data_mining_2/insurance_cleaned_missing.csv")

# Define continuous columns from updated structure
continuous_columns = [
    "total_properties_owned", "avg_household_size", "childless_households",
    "households_with_children", "married", "living_together", "extended_family_members", "singles",
    "highly_educated", "moderately_educated", "low_education",
    "high_status_professionals", "entrepreneurs", "farmer", "middle_management",
    "skilled_workers", "unskilled_workers",
    "social_class_a", "social_class_b1", "social_class_b2",
    "social_class_c", "social_class_d",
    "rented_house", "home_owner",
    "single_car", "dual_car", "no_car",
    "income_below_30k", "income_30k_to_45k", "income_45k_to_75k",
    "income_75k_to_122k", "income_above_123k", "mean_income", "purchasing_power_group",
    "car_insurance", "van_insurance", "bike_insurance",
    "lorry_insurance", "trailer_insurance", "tractor_insurance",
    "agrimachine_insurance", "moped_insurance",
    "life_insurance", "private_accident_insurance", "family_accident_insurance",
    "disability_insurance", "fire_insurance", "property_insurance",
    "social_security_insurance",
    "private_third_party_policies_count", "business_third_party_policies_count",
    "agricultural_third_party_policies_count",
    "car_policy_count", "van_policy_count", "bike_policy_count",
    "lorry_policy_count", "trailer_policy_count", "tractor_policy_count",
    "agrimachine_policy_count", "moped_policy_count",
    "life_insurance_count", "personal_accident_count",
    "family_accident_count", "disability_insurance_count",
    "fire_insurance_count", "property_insurance_count",
    "social_security_count", "mobile_home_policy_count"
]

# Function to detect outliers using the IQR method
def detect_outliers(data, columns):
    """
    Detects outliers in specified columns using the IQR method.
```

```
    Returns a dictionary with outlier counts, percentages, and indices.
    """
    outlier_dict = {}
    for col in columns:
        if col not in ins2.columns or not pd.api.types.is_numeric_dtype(ins2[col]):
            continue
        Q1 = data[col].quantile(0.25)
        Q3 = data[col].quantile(0.75)
        IQR = Q3 - Q1
        lower_bound = round(Q1 - 1.5 * IQR, 1)
        upper_bound = round(Q3 + 1.5 * IQR, 1)
        outliers = data[(data[col] < lower_bound) | (data[col] > upper_bound)]
        if not outliers.empty:
            outlier_dict[col] = {
                'count': len(outliers),
                'percent': round((len(outliers) / len(data)) * 100, 1),
                'lower_bound': round(lower_bound),
                'upper_bound': round(upper_bound)
            }
    return outlier_dict

# Detect outliers
outlier_dict = detect_outliers(ins2, continuous_columns)

# Convert all detected outliers to a DataFrame
if outlier_dict:
    outlier_table = pd.DataFrame.from_dict(outlier_dict, orient='index')
    outlier_table.index.name = 'Variable'
    outlier_table.reset_index(inplace=True)
    outlier_table.rename(columns={
        'count': 'Outlier Count',
        'percent': 'Percentage (%)',
        'lower_bound': 'Lower Bound',
        'upper_bound': 'Upper Bound'
    }, inplace=True)

    # Format the 'Percentage (%)' column to 1 decimal place and add '%'
    outlier_table['Percentage (%)'] = outlier_table['Percentage (%)'].apply(lambda x: f"{x:.1f}%")

    # Sort by Outlier Count in descending order
    outlier_table = outlier_table.sort_values(by='Outlier Count', ascending=False)

    # Display the table
    from IPython.display import display
    display(outlier_table.style.hide(axis="index"))
else:
    print("No outliers detected in the dataset.")
```

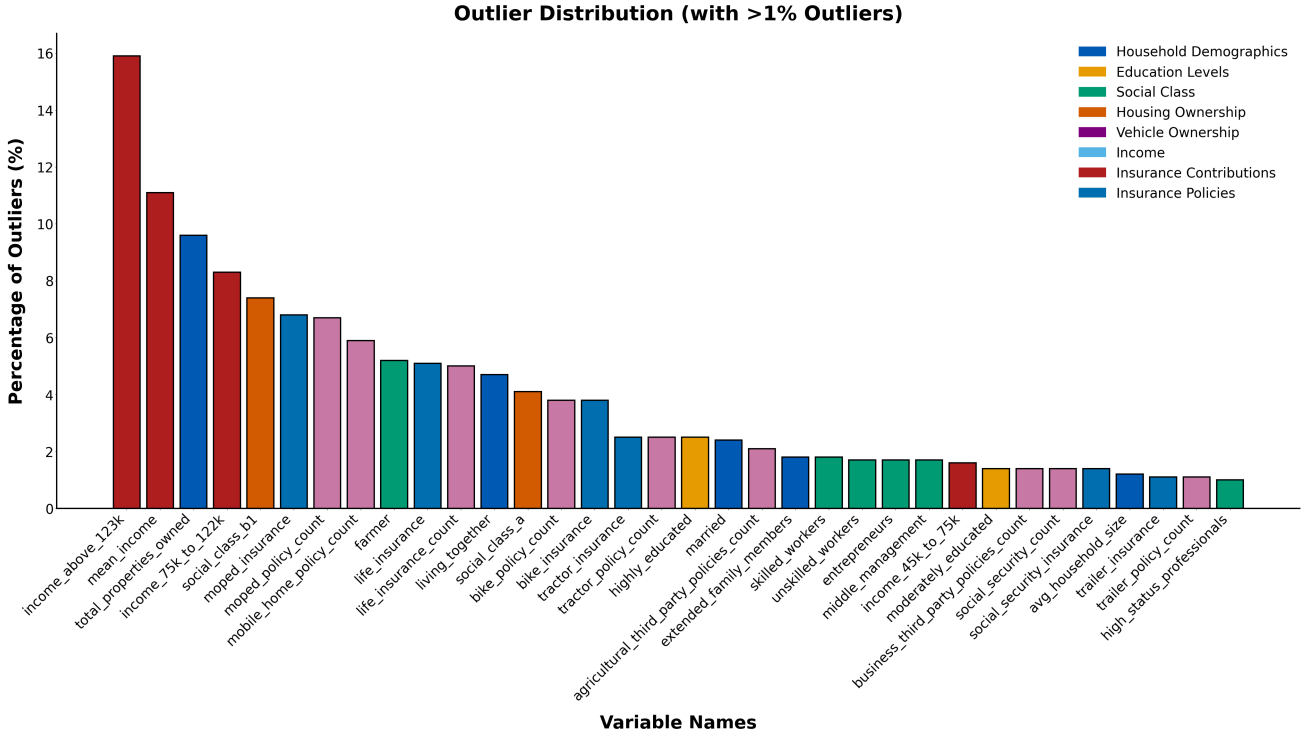
Variable	Outlier Count	Percentage (%)	Lower Bound	Upper Bound
income_above_123k	879	15.9%	0	0
mean_income	615	11.1%	2	6
total_properties_owned	528	9.6%	1	1
income_75k_to_122k	461	8.3%	-2	2
social_class_b1	407	7.4%	0	4
moped_insurance	373	6.8%	0	0
moped_policy_count	372	6.7%	0	0
mobile_home_policy_count	328	5.9%	0	0
farmer	289	5.2%	-2	2
life_insurance	279	5.1%	0	0
life_insurance_count	278	5.0%	0	0
living_together	257	4.7%	-2	2
social_class_a	229	4.1%	-3	5
bike_policy_count	209	3.8%	0	0
bike_insurance	209	3.8%	0	0
tractor_insurance	139	2.5%	0	0
tractor_policy_count	139	2.5%	0	0
highly_educated	138	2.5%	-3	5
married	130	2.4%	2	10
agricultural_third_party_policies_count	116	2.1%	0	0
extended_family_members	100	1.8%	-2	6
skilled_workers	97	1.8%	-2	6
unskilled_workers	96	1.7%	-2	6
entrepreneurs	95	1.7%	-2	2
middle_management	92	1.7%	-1	7
income_45k_to_75k	90	1.6%	-4	8
moderately_educated	78	1.4%	-1	7
business_third_party_policies_count	78	1.4%	0	0
social_security_count	77	1.4%	0	0
social_security_insurance	77	1.4%	0	0
avg_household_size	65	1.2%	0	4
trailer_insurance	60	1.1%	0	0
trailer_policy_count	60	1.1%	0	0
high_status_professionals	53	1.0%	-4	8
no_car	50	0.9%	-2	6
van_policy_count	48	0.9%	0	0
van_insurance	48	0.9%	0	0
income_below_30k	48	0.9%	-4	8
property_insurance_count	44	0.8%	0	0
property_insurance	44	0.8%	0	0
childless_households	43	0.8%	-1	7
singles	42	0.8%	-4	8
family_accident_count	33	0.6%	0	0
family_accident_insurance	33	0.6%	0	0
social_class_d	32	0.6%	-3	5
single_car	30	0.5%	2	10
private_accident_insurance	29	0.5%	0	0
personal_accident_count	29	0.5%	0	0
disability_insurance_count	23	0.4%	0	0
disability_insurance	22	0.4%	0	0
agrimachine_policy_count	21	0.4%	0	0
agrimachine_insurance	21	0.4%	0	0
car_policy_count	18	0.3%	-2	2
social_class_b2	14	0.3%	-2	6
fire_insurance_count	12	0.2%	-2	2
dual_car	10	0.2%	-3	5
lorry_insurance	9	0.2%	0	0
lorry_policy_count	9	0.2%	0	0
fire_insurance	4	0.1%	-6	10

4.1.2 Visualise IQR outliers

In [116…

```
# Define variable groupings for plots
groups = {
    "household_demographics": ["total_properties_owned", "avg_household_size", "childless_households", "households_with_children",
                               "married", "living_together", "extended_family_members", "singles"],

    "education_levels": ["highly_educated", "moderately_educated", "low_education"],

    "occupation_types": ["high_status_professionals", "entrepreneurs", "farmer",
                         "middle_management", "skilled_workers", "unskilled_workers"],

    "social_class": ["social_class_a", "social_class_b1", "social_class_b2", "social_class_c", "social_class_d"],

    "housing_ownership": ["rented_house", "home_owner"],

    "vehicle_ownership": ["single_car", "dual_car", "no_car"],

    "income_vars": ["income_below_30k", "income_30k_to_45k", "income_45k_to_75k",
                    "income_75k_to_122k", "income_above_123k", "mean_income", "purchasing_power_group"],

    "financial": ["car_insurance", "van_insurance", "bike_insurance", "lorry_insurance",
                  "trailer_insurance", "tractor_insurance", "agrimachine_insurance", "moped_insurance",
                  "life_insurance", "private_accident_insurance", "family_accident_insurance",
                  "disability_insurance", "fire_insurance", "property_insurance", "social_security_insurance"],
```

```
    "insurance": ["private_third_party_policies_count", "business_third_party_policies_count",
                  "agricultural_third_party_policies_count", "car_policy_count", "van_policy_count",
                  "bike_policy_count", "lorry_policy_count", "trailer_policy_count", "tractor_policy_count",
                  "agrimachine_policy_count", "moped_policy_count", "life_insurance_count",
                  "personal_accident_count", "family_accident_count", "disability_insurance_count",
                  "fire_insurance_count", "property_insurance_count", "social_security_count",
                  "mobile_home_policy_count"]
    }


# Define color mapping for each group with color blindness-friendly shades
group_colors = {
    "household_demographics": "#005AB5",  # Deep blue
    "education_levels": "#E69F00",  # Golden yellow
    "occupation_types": "#009E73",  # Muted teal green
    "social_class": "#D55E00",  # Rust orange
    "housing_ownership": "#800080",  # Purple
    "vehicle_ownership": "#56B4E9",  # Sky blue
    "income_vars": "#B22222",  # Deep red
    "financial": "#0072B2",  # Strong blue
    "insurance": "#CC79A7"  # Soft purple/magenta
    }

# Filter variables with more than 1% outliers
filtered_outlier_table = outlier_table[outlier_table["Percentage (%)"].str.rstrip('%').astype(float) >= 1]

# Extract variable names and outlier percentages
variables = filtered_outlier_table["Variable"]
percentages = filtered_outlier_table["Percentage (%)"].str.rstrip('%').astype(float)  # Convert percentages to numeric

# Assign colors to each variable based on its grouping
variable_colors = [
    next((color for group, color in group_colors.items() if var in groups[group]), "black")
    for var in variables
]

# Create bar plot with improved aesthetics
plt.figure(figsize=(50, 20))
plt.bar(variables, percentages, color=variable_colors, edgecolor="black", linewidth=3, alpha=1)

# Set axis title font sizes
plt.xlabel("Variable Names", fontsize=40, fontweight='bold', labelpad=25)
plt.ylabel("Percentage of Outliers (%)", fontsize=40, fontweight='bold', labelpad=25)
plt.title("Outlier Distribution (with >1% Outliers)",
          fontsize=45, fontweight='bold', pad=30)

# Set axis tick sizes and ensure y-axis ticks are displayed
plt.xticks(rotation=45, ha="right", fontsize=30)
plt.yticks(fontsize=30)  # Explicitly set y-axis tick size

# Remove gridlines completely by setting a white background
plt.gca().set_facecolor("white")
plt.grid(False)

# Improve axis aesthetics
plt.gca().spines["top"].set_visible(False)
plt.gca().spines["right"].set_visible(False)
plt.gca().spines["left"].set_linewidth(3)
plt.gca().spines["bottom"].set_linewidth(3)

# Show legend with improved figure description
handles = [plt.Rectangle((0, 0), 1, 1, color=color, edgecolor="black") for color in group_colors.values()]
legend_labels = [
    "Household Demographics", "Education Levels", "Social Class",
    "Housing Ownership", "Vehicle Ownership", "Income",
    "Insurance Contributions", "Insurance Policies"
]
plt.legend(handles, legend_labels, fontsize=30, frameon=False, loc="upper right")
plt.subplots_adjust(top=0.85) # Optimize spacing between elements
plt.show()

print('')
print('')
print('')
```



Dominant Outlier Categories: Income-related variables demonstrate the highest concentration of outliers, with "income_above_125k" showing nearly 16% outliers and "mean_income" at approximately 11%. This suggests substantial income inequality or polarization within the sample, where extreme high earners create significant deviations from central tendencies.

Demographic and Socioeconomic Patterns: Total properties owned (9.7%) and several education-related variables (8.4% for "income_25k_125k" and 7.4% for "social_class_ul") exhibit elevated outlier rates. The presence of outliers in both income and education variables indicates potential clustering of socioeconomic extremes, likely representing both highly advantaged and disadvantaged population segments.

Insurance and Policy Variables: Multiple insurance-related variables show moderate outlier concentrations (4-7%), suggesting heterogeneous coverage patterns or premium structures that deviate significantly from typical population characteristics. This may reflect specialized insurance products, high-risk categories, or geographic variations in coverage.

Geographic and Housing Stability: Lower outlier percentages in variables like "avg_household_size," "trailer_insurance," and several policy-related measures (1-2%) suggest these characteristics follow more normal distributions with fewer extreme cases.

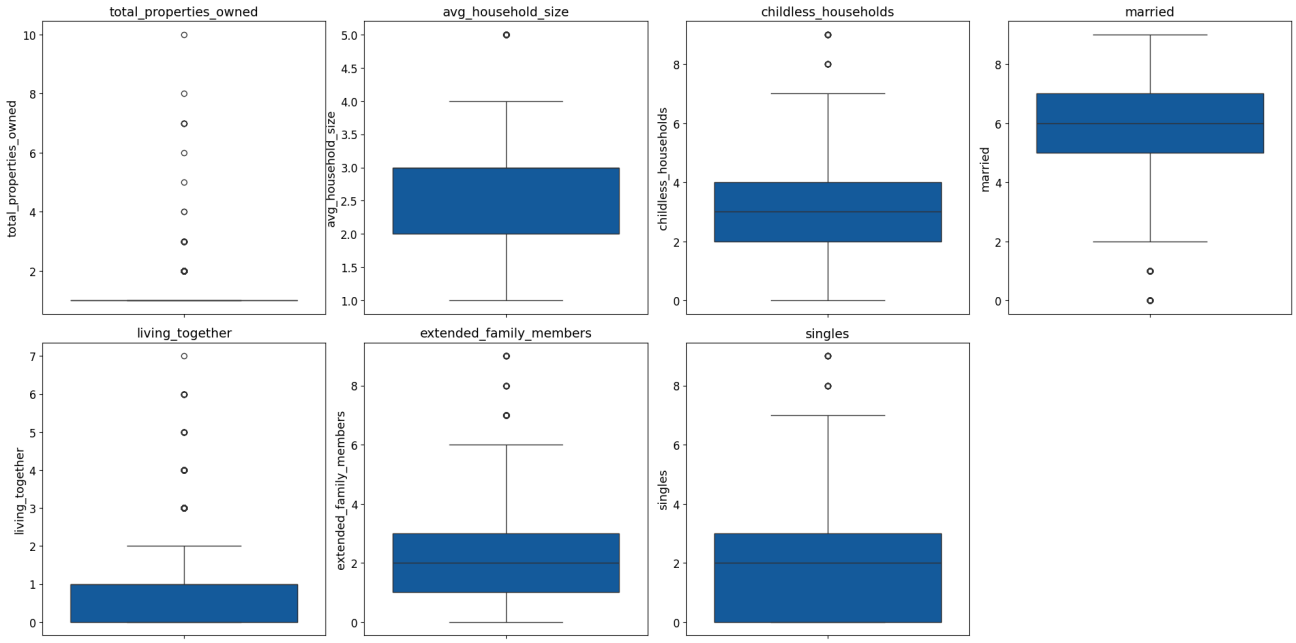
Methodological Implications: The >1% threshold filter effectively captures meaningful outliers while excluding trivial deviations. The predominance of income and socioeconomic outliers suggests the dataset may benefit from stratified analysis or robust statistical methods that account for these distributional irregularities, particularly when modeling relationships involving economic variables.

4.1.3 Visualise Outliers as Boxplots

In [1]:
```python
def plot_boxplots(data, columns, group_name, figsize=(20, 15), ncols=2, wspace=0.5, hspace=0.5):
    """
    Creates boxplots for specified columns, color-coded by group.

    Parameters:
    - data: DataFrame containing the data.
    - columns: List of columns to plot.
    - group_name: Name of the variable group to assign color.
    - figsize: Tuple specifying the figure size.
    - ncols: Number of columns in the subplot grid.
    - wspace: Width spacing between plots.
    - hspace: Height spacing between plots.
    """
    # Filter numeric columns that exist in the dataframe
    valid_cols = [col for col in columns if col in data.columns and pd.api.types.is_numeric_dtype(data[col])]

    # Get color for the group
    plot_color = group_colors.get(group_name, "#000000")  # Default to black if group is missing

    # Calculate number of rows needed
    nrows = int(np.ceil(len(valid_cols) / ncols))

    fig, axes = plt.subplots(nrows=nrows, ncols=ncols, figsize=figsize)
    axes = axes.flatten()

    for i, col in enumerate(valid_cols):
        sns.boxplot(y=data[col], ax=axes[i], color=plot_color)
        axes[i].set_title(f'{col}', fontsize=14)
        axes[i].set_ylabel(col, fontsize=13)
        axes[i].tick_params(axis='both', labelsize=12)

    # Hide unused subplots
    for j in range(len(valid_cols), len(axes)):
        fig.delaxes(axes[j])

    # Adjust spacing between plots
    plt.subplots_adjust(wspace=wspace, hspace=hspace)
    plt.tight_layout()
    plt.show()
```

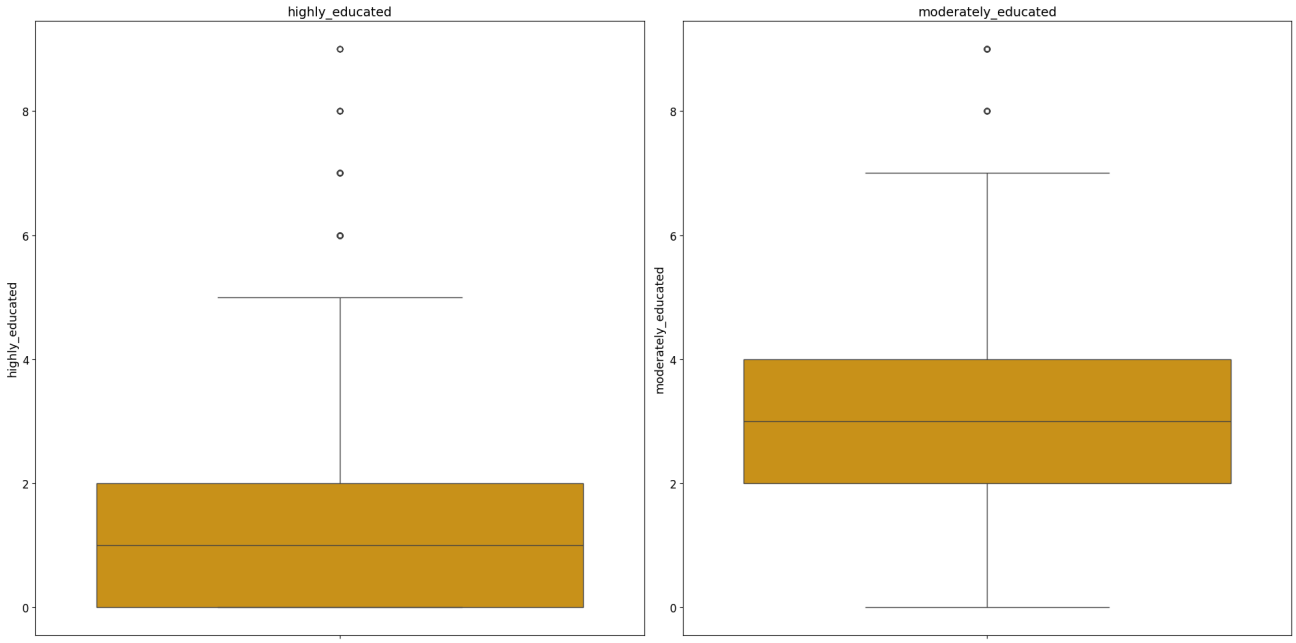
4.1.3a Household Demographics

In [9]:
```python
filtered_household_vars = [var for var in groups["household_demographics"] if var in outlier_table["Variable"].values]
if filtered_household_vars:
    plot_boxplots(ins2, filtered_household_vars, "household_demographics", figsize=(20, 10), ncols=min(len(filtered_household_vars), 4), wspace=5, hspace=5)
else:
    print("No outliers were detected for Household Demographics.")
```



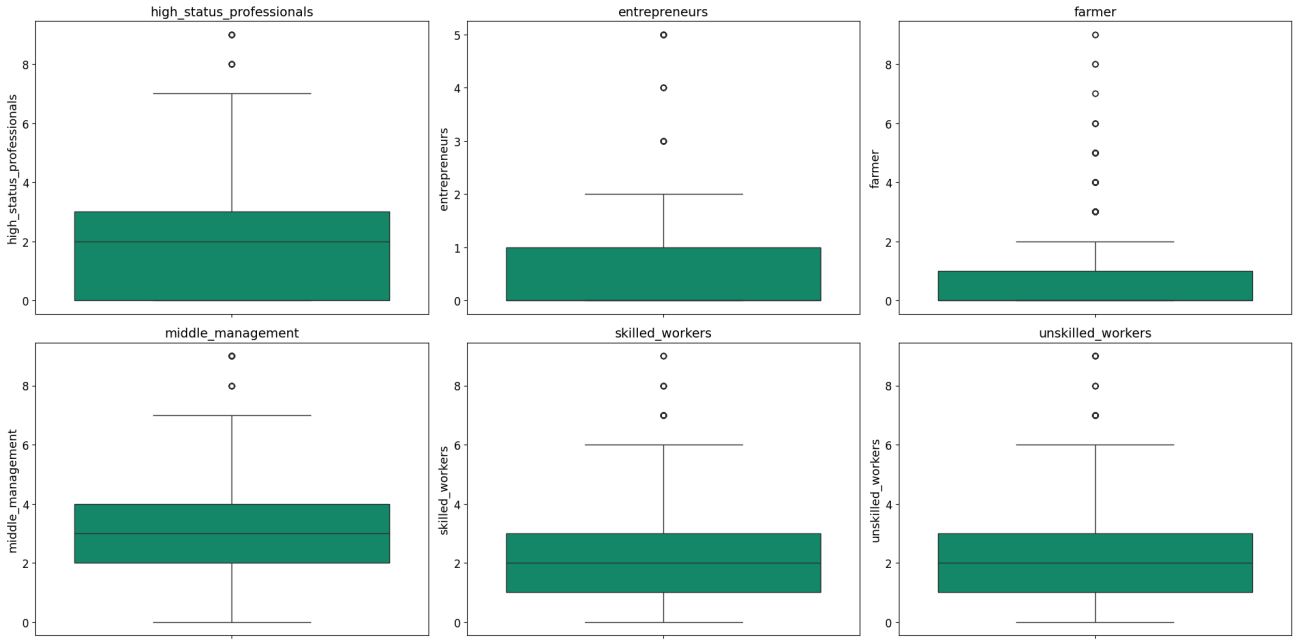
4.1.3b Education Levels

In [10]:
```python
filtered_education_vars = [var for var in groups["education_levels"] if var in outlier_table["Variable"].values]
if filtered_education_vars:
    plot_boxplots(ins2, filtered_education_vars, "education_levels", figsize=(20, 10), ncols=min(len(filtered_education_vars), 3), wspace=5, hspace=5)
else:
    print("No outliers were detected for Education Levels.")
```



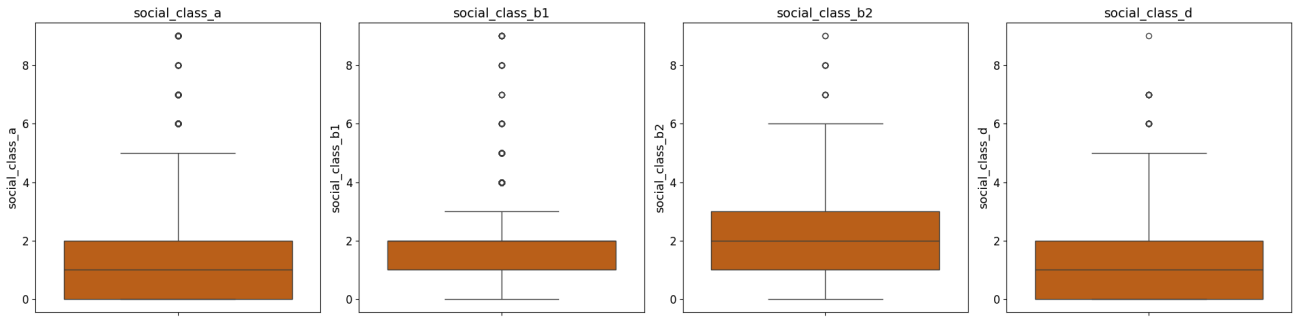
4.1.3c Occupation Types

In [11]:
```
filtered_occupation_vars = [var for var in groups["occupation_types"] if var in outlier_table["Variable"].values]
if filtered_occupation_vars:
    plot_boxplots(ins2, filtered_occupation_vars, "occupation_types", figsize=(20, 10), ncols=min(len(filtered_occupation_vars), 3), wspace=5, hspace=5)
else:
    print("No outliers were detected for Occupation Types.")
```



4.1.3d Social Class

In [12]:
```
filtered_social_class_vars = [var for var in groups["social_class"] if var in outlier_table["Variable"].values]
if filtered_social_class_vars:
    plot_boxplots(ins2, filtered_social_class_vars, "social_class", figsize=(20, 5), ncols=min(len(filtered_social_class_vars), 5), wspace=5, hspace=5)
else:
    print("No outliers were detected for Social Class.")
```



4.1.3e Housing Ownership

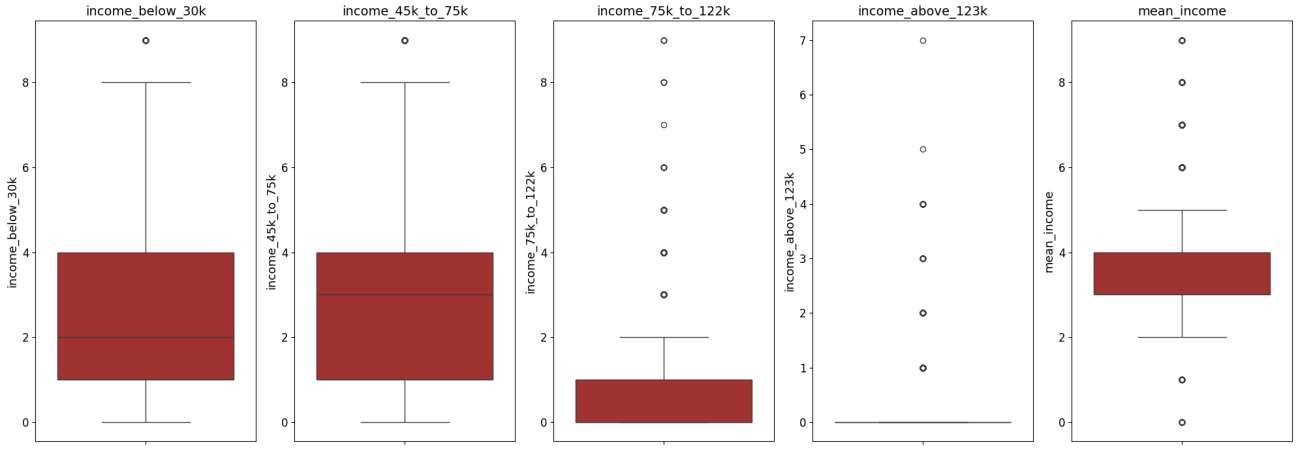
In [13]:
```
filtered_housing_vars = [var for var in groups["housing_ownership"] if var in outlier_table["Variable"].values]
if filtered_housing_vars:
    plot_boxplots(ins2, filtered_housing_vars, "housing_ownership", figsize=(10, 5), ncols=min(len(filtered_housing_vars), 2), wspace=5, hspace=5)
```

```
else:
    print("No outliers were detected for Housing Ownership.")
```

```
No outliers were detected for Housing Ownership.
```

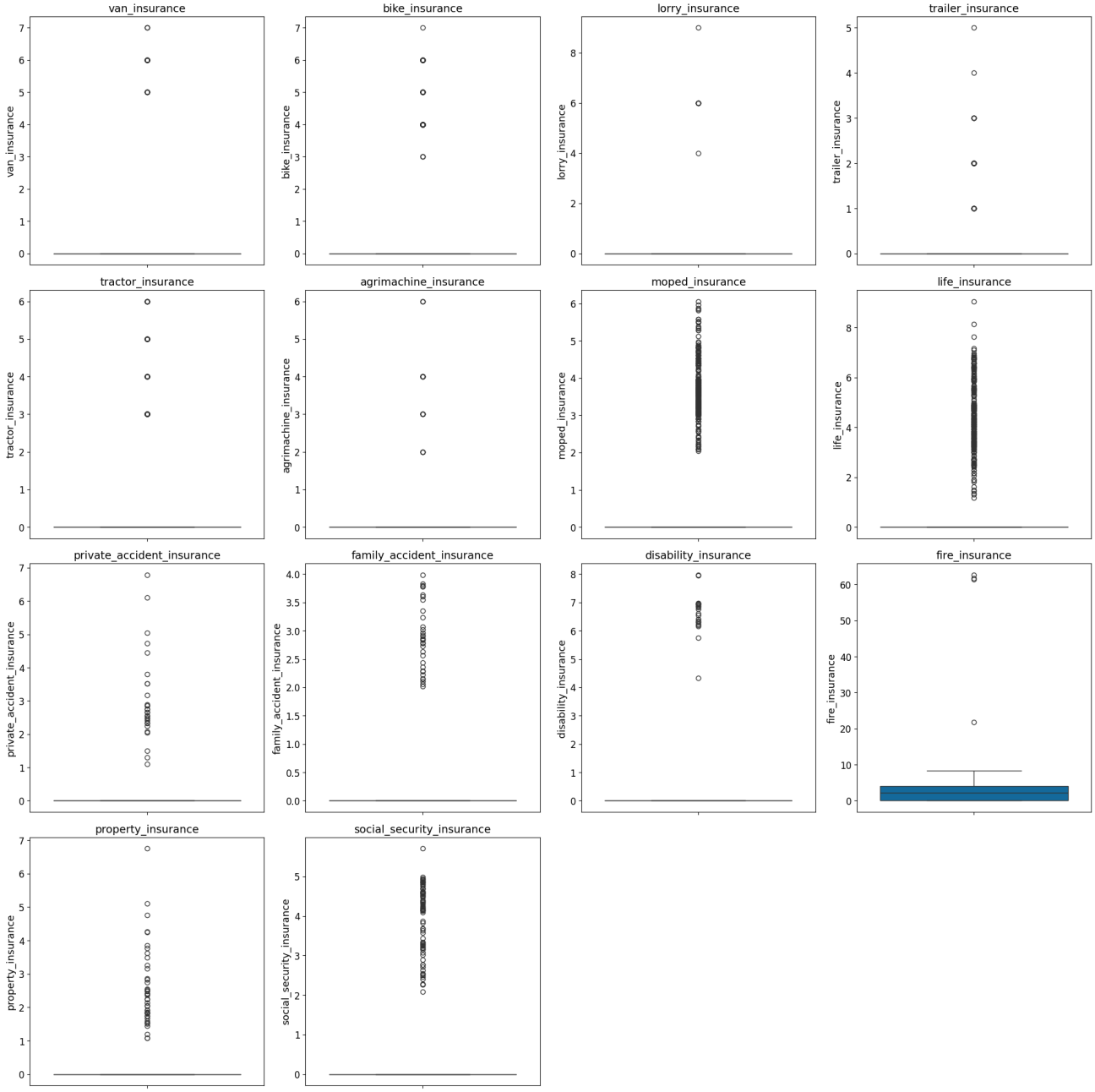
4.1.3f Income

In [14]:
```
filtered_income_vars = [var for var in groups["income_vars"] if var in outlier_table["Variable"].values]
if filtered_income_vars:
    plot_boxplots(ins2, filtered_income_vars, "income_vars", figsize=(20, 7), ncols=min(len(filtered_income_vars), 5), wspace=5, hspace=5)
else:
    print("No outliers were detected for Income Variables.")
```



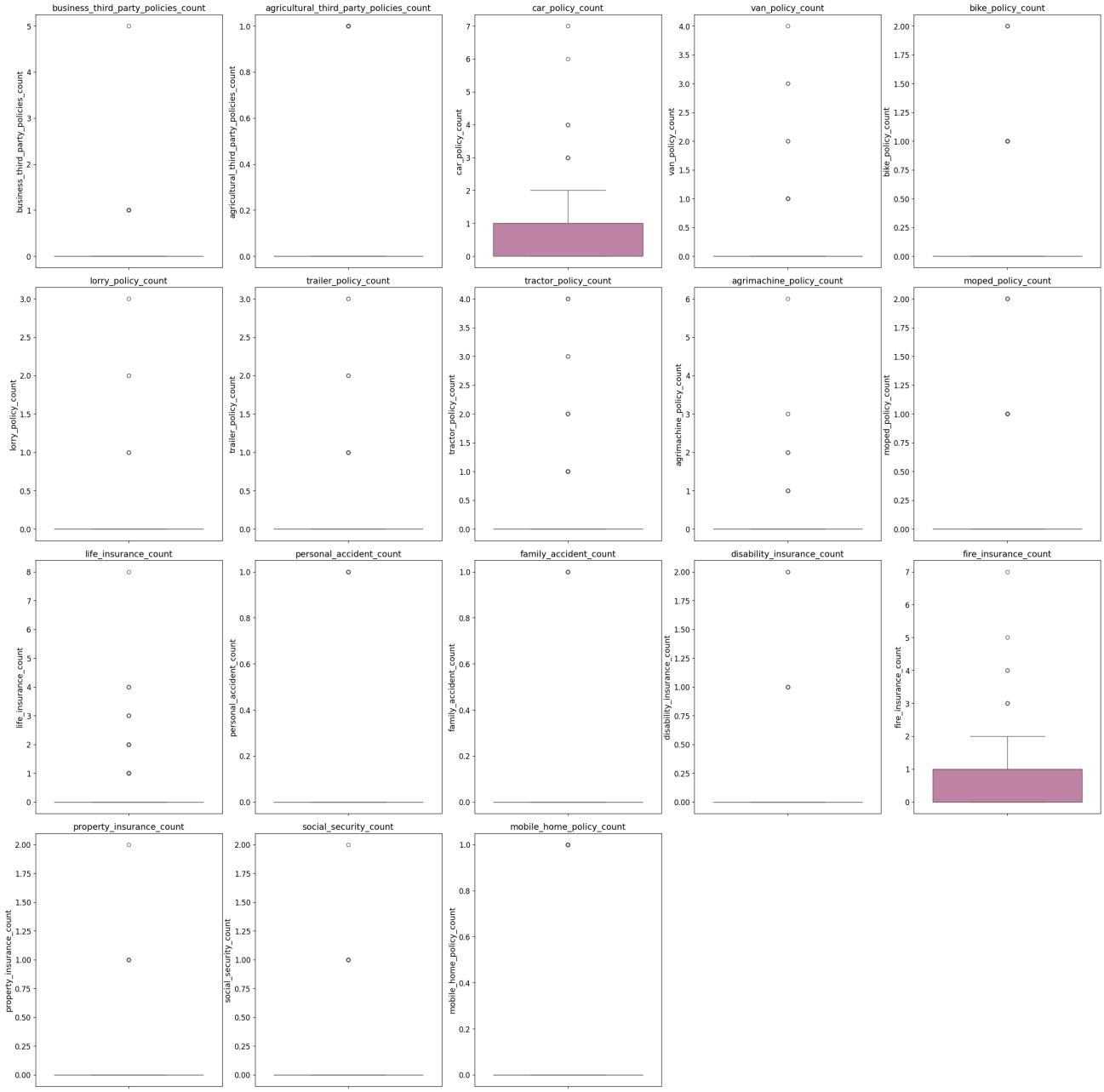
4.1.3g Financial

In [15]:
```
filtered_financial_vars = [var for var in groups["financial"] if var in outlier_table["Variable"].values]
if filtered_financial_vars:
    plot_boxplots(ins2, filtered_financial_vars, "financial", figsize=(20, 20), ncols=min(len(filtered_financial_vars), 4), wspace=5, hspace=5)
else:
    print("No outliers were detected for Financial Metrics.")
```



4.1.3h Insurance Policies

In [16]:
```
filtered_insurance_vars = [var for var in groups["insurance"] if var in outlier_table["Variable"].values]
if filtered_insurance_vars:
    plot_boxplots(ins2, filtered_insurance_vars, "insurance", figsize=(25, 25), ncols=min(len(filtered_insurance_vars), 5), wspace=5, hspace=5)
else:
    print("No outliers were detected for Insurance Policies.")
```



4.2 Outlier Handling - Sampling (train/validation/test)

In [63]:
```
# Train-test split preparation
from sklearn.model_selection import train_test_split

# Define X (continuous variables) and y (target variable)
X = ins2[continuous_columns]
y = ins2["customer_type"]

# Split data
print("\n=== Data Splitting ===")
part1_X, test_X, part1_y, test_y = train_test_split(X, y, test_size=0.2, random_state=1, stratify=y)
train_X, val_X, train_y, val_y = train_test_split(part1_X, part1_y, test_size=0.2, random_state=1)

print(f"Training set: {train_X.shape[0]} samples")
print(f"Validation set: {val_X.shape[0]} samples")
print(f"Test set: {test_X.shape[0]} samples")
print(f"Insurance: {ins2.shape[0]} samples") # checking total
```

```
=== Data Splitting ===
Training set: 3532 samples
Validation set: 884 samples
Test set: 1105 samples
Insurance: 5521 samples
```

4.3 Data preparation for Supervised and Unsupervised Learning

Applying Unsupervised Learning (DBSCAN) Before Supervised Classification

Unsupervised learning methods, such as **DBSCAN** (Density-Based Spatial Clustering of Applications with Noise), play a crucial role in data exploration before applying supervised classification. This methodological approach allows us to understand the natural structure of the dataset without predefined labels (`Customer_Type`). In the insurance domain, where customer profiling is vital for business strategies, DBSCAN clustering enables segmentation based purely on feature similarities rather than manually assigned categories.

Why Begin with DBSCAN Clustering?

1. Exploratory Data Analysis (EDA) without Labels

Unsupervised learning does not rely on existing classifications. It autonomously discovers meaningful patterns and segments that may not be immediately apparent in predefined `Customer_Type` labels.

- DBSCAN is ideal when the true number of clusters is unknown, making it a flexible tool for insurance customer segmentation.

2. Handling High-Dimensional, Complex Data

- Our dataset comprises **5,521 observations and 83 continuous features**, representing various socioeconomic and insurance-related attributes.
- Traditional clustering methods like `K-Means` assume spherical clusters, while DBSCAN is **density-based**, allowing irregular shapes suited for real-world customer distributions.

3. Detection of Outliers and Noise

- Unlike supervised learning, DBSCAN explicitly **identifies outliers** (points labelled as `-1`). These outliers may represent rare customer profiles, fraud cases, or individuals who do not conform to typical segmentation patterns.
- Understanding these outliers is essential before refining models for **supervised classification**.

4. Improved Feature Engineering for Supervised Models

- The clusters formed by DBSCAN can be used to **enhance classification** by introducing additional engineered features (e.g., cluster membership as a categorical predictor), improving **interpretability**, allowing domain experts to derive insights before implementing supervised models.

4.4 DBSCAN

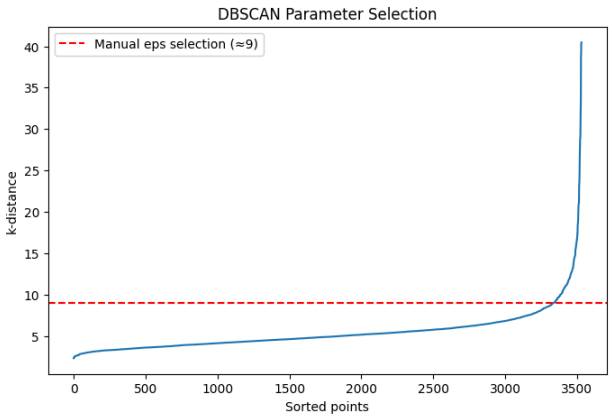
[Disclaimer]: The following code for Data Dictionary generation was supported by Microsoft Copilot/Claude.ai, an AI-powered assistant to assist in Python code generation and debugging.

4.4.1 DBSCAN Implementation and Parameter Optimization

Finding the optimal parameters for DBSCAN is crucial for effective outlier detection. The function `find_eps_second_derivative` implements the k-distance graph method to determine appropriate epsilon `eps` and `minimum samples` parameters for DBSCAN. The k-distance graph provides an empirical approach to determining appropriate density thresholds for identifying clusters and outliers, valuable in the context of complex customer data, where the boundary between normal variation and genuine outliers may not be immediately apparent. By optimizing these parameters, we improve our ability to identify meaningful customer segments and detect anomalous profiles that may require special attention in our classification models.

Commencing with the requisite library imports, including StandardScaler for feature normalization—a critical preprocessing step for distance-based methods—and DBSCAN for clustering, the workflow proceeds to standardize the continuous variables within the training dataset to mitigate scale-dependent biases. A central component of this process involves the rigorous determination of the optimal eps parameter for DBSCAN, achieved through two distinct methodologies: an automated technique employing the second derivative to identify the point of maximal curvature in the k-distance graph, and a manual approach facilitating visual inspection and selection from the same graph. A comparative analysis of the eps values suggested by these methods is conducted. Subsequently, a dedicated function is defined to operationalize the DBSCAN algorithm with specified eps and min_samples values, providing a quantitative summary of detected clusters and identified outliers. The segment concludes by applying DBSCAN using both the automatically and manually derived eps values, in conjunction with a min_samples parameter heuristically set as twice the number of features, thereby allowing for an empirical evaluation and comparison of the outlier detection efficacy under different eps selections.

In [64]:
```python
#Import Required Libraries
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import NearestNeighbors
from sklearn.cluster import DBSCAN

# Standardize Continuous Features
# DBSCAN is sensitive to feature scale. select continuous variables** from our dataset and apply **standardization** to normalize their values.
X_scaled = StandardScaler().fit_transform(train_X[continuous_columns])

# Determine Optimal Eps for DBSCAN [See Disclaimer]
# Method 1: Automated Selection Using Second Derivative to find the elbow point in the k-distance graph by detecting the highest rate of change in distances.

def find_eps_second_derivative(X_scaled): # [See Disclaimer]
    """
    Automated selection using the second derivative method.

    Args:
        X_scaled: Scaled feature matrix.

    Returns:
        Suggested eps value.
    """
    distances = np.sort(NearestNeighbors(n_neighbors=10).fit(X_scaled).kneighbors(X_scaled)[0][:, -1])
    return distances[np.argmax(np.diff(np.diff(distances)))]  # Finding max curvature point

# Method 2: Manual Selection Based on Graph Visualization to visually inspect where the elbow occurs and choose eps manually. [See Disclaimer]

def find_eps_visual(X_scaled):
    """
    Manual selection based on k-distance graph visualization.

    Args:
        X_scaled: Scaled feature matrix.

    Returns:
        Manually selected eps value.
    """
    distances = np.sort(NearestNeighbors(n_neighbors=10).fit(X_scaled).kneighbors(X_scaled)[0][:, -1])

    # Plot k-distance graph for visual inspection
    plt.figure(figsize=(8, 5))
    plt.plot(distances)
    plt.axhline(y=9, color='r', linestyle='--', label='Manual eps selection (=9)')
    plt.xlabel("Sorted points")
    plt.ylabel("k-distance")
    plt.title("DBSCAN Parameter Selection")
    plt.legend()
    plt.show()

    return 9  # Manually chosen based on observed elbow point

# Compare Both Methods by displaying suggested `eps` values for DBSCAN.

eps_auto = find_eps_second_derivative(X_scaled)
eps_manual = find_eps_visual(X_scaled)

print(f"Automated eps (Second Derivative): {eps_auto:.3f}")
print(f"Manual eps (Visual Selection): {eps_manual:.3f}")

# Apply DBSCAN Clustering with both selected `eps` values to analyze the number of clusters and outliers.

def detect_outliers_with_dbscan(X_scaled, eps, min_samples): # [See Disclaimer]
    """
    Applies DBSCAN clustering to identify outliers.

    Args:
        X_scaled: Scaled feature matrix.
        eps: Epsilon radius for DBSCAN.
        min_samples: Minimum points required for a cluster.

    Returns:
        Cluster labels (-1 for outliers).
    """
    clusters = DBSCAN(eps=eps, min_samples=min_samples).fit_predict(X_scaled)
    n_clusters = len(set(clusters)) - (1 if -1 in clusters else 0)
    n_noise = list(clusters).count(-1)

    print(f"DBSCAN Parameters: eps={eps:.3f}, min_samples={min_samples}")
    print(f"Clusters: {n_clusters}, Outliers: {n_noise} ({n_noise/len(X_scaled)*100:.2f}%)")

    return clusters

# Run DBSCAN with Both Eps Values. Calculate `min_samples` using a heuristic (`2 * number of features`) and compare the clustering results.

min_samples = 2 * X_scaled.shape[1]  # Heuristic for DBSCAN min_samples

print("\n# Using Automated eps:")
outlier_labels_auto = detect_outliers_with_dbscan(X_scaled, eps_auto, min_samples)

print("\n# Using Manual eps:")
outlier_labels_manual = detect_outliers_with_dbscan(X_scaled, eps_manual, min_samples)
```



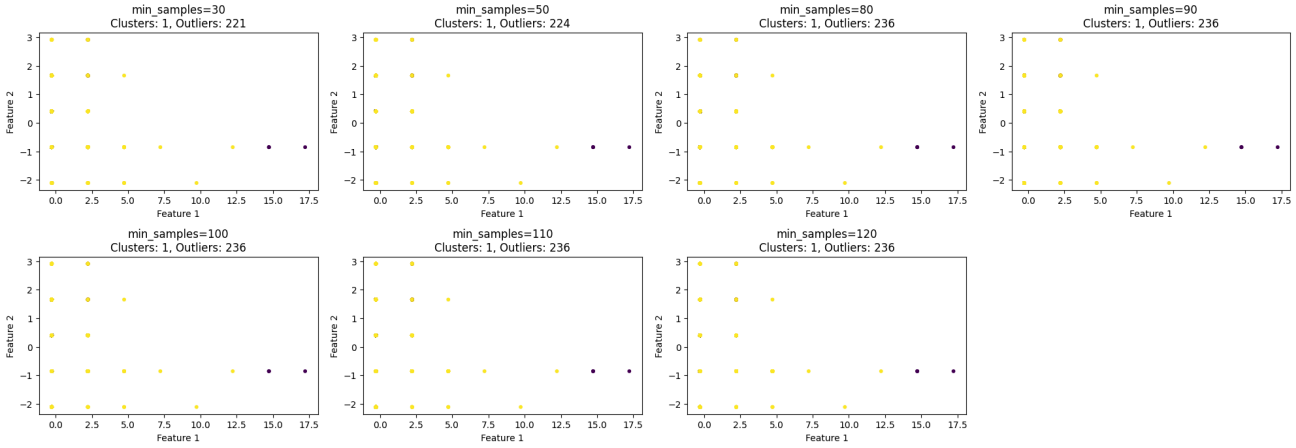

```
Automated eps (Second Derivative): 32.404
Manual eps (Visual Selection): 9.000

# Using Automated eps:
DBSCAN Parameters: eps=32.404, min_samples=136
Clusters: 1, Outliers: 2 (0.06%)

# Using Manual eps:
DBSCAN Parameters: eps=9.000, min_samples=136
Clusters: 1, Outliers: 237 (6.71%)
```

4.4.2 Sensitivity Analysis for DBSCAN Clustering [See Disclaimer]

The following code systematically evaluates the influence of the `min_samples` parameter on DBSCAN clustering. The **Density-Based Spatial Clustering of Applications with Noise (DBSCAN)** algorithm was employed using varying values for the `min_samples` parameter to analyze customer segmentation within the dataset. The primary objective of this analysis was to identify distinct clusters while simultaneously detecting outliers that may represent unique or atypical customer profiles.

In [65]:
```python
# DBSCAN Implementation
from collections import defaultdict

# Define range of min_samples values for sensitivity analysis
min_samples_values = [30, 50] + list(range(80, 130, 10))  # Exploring cluster stability

results = defaultdict(dict)  # Store clustering outcomes

plt.figure(figsize=(20, 10))

for i, min_samples in enumerate(min_samples_values, 1):
    # Apply DBSCAN with chosen eps and min_samples values
    clusters = DBSCAN(eps=9, min_samples=min_samples).fit_predict(X_scaled)

    # Identify number of clusters and outliers
    n_clusters = len(set(clusters)) - (1 if -1 in clusters else 0)  # Adjust for noise
    n_noise = list(clusters).count(-1)  # Outliers labeled as -1

    results[min_samples]["clusters"] = n_clusters
    results[min_samples]["outliers"] = n_noise
    results[min_samples]["outlier_percentage"] = (n_noise / len(X_scaled)) * 100

    print(f"\n# DBSCAN with min_samples={min_samples}:")
    print(f"Clusters Detected: {n_clusters}, Outliers Identified: {n_noise} ({results[min_samples]['outlier_percentage']:.2f}%)")

    # Visualizing clusters (first two features) to observe segmentation
    plt.subplot(3, 4, i)
    plt.scatter(X_scaled[:, 0], X_scaled[:, 1], c=clusters, cmap='viridis', s=10)
    plt.title(f"min_samples={min_samples}\nClusters: {n_clusters}, Outliers: {n_noise}")
    plt.xlabel("Feature 1")
    plt.ylabel("Feature 2")

plt.tight_layout()
plt.show()

# Display summary of clustering results
print("\n### Summary of DBSCAN Results:")
for min_samples, data in results.items():
    print(f"min_samples={min_samples}: {data}")
```

```
# DBSCAN with min_samples=30:
Clusters Detected: 1, Outliers Identified: 221 (6.26%)

# DBSCAN with min_samples=50:
Clusters Detected: 1, Outliers Identified: 224 (6.34%)

# DBSCAN with min_samples=80:
Clusters Detected: 1, Outliers Identified: 236 (6.68%)

# DBSCAN with min_samples=90:
Clusters Detected: 1, Outliers Identified: 236 (6.68%)

# DBSCAN with min_samples=100:
Clusters Detected: 1, Outliers Identified: 236 (6.68%)

# DBSCAN with min_samples=110:
Clusters Detected: 1, Outliers Identified: 236 (6.68%)

# DBSCAN with min_samples=120:
Clusters Detected: 1, Outliers Identified: 236 (6.68%)
```




```
### Summary of DBSCAN Results:
min_samples=30: {'clusters': 1, 'outliers': 221, 'outlier_percentage': 6.257078142695356}
min_samples=50: {'clusters': 1, 'outliers': 224, 'outlier_percentage': 6.342015855039637}
min_samples=80: {'clusters': 1, 'outliers': 236, 'outlier_percentage': 6.681766704416761}
min_samples=90: {'clusters': 1, 'outliers': 236, 'outlier_percentage': 6.681766704416761}
min_samples=100: {'clusters': 1, 'outliers': 236, 'outlier_percentage': 6.681766704416761}
min_samples=110: {'clusters': 1, 'outliers': 236, 'outlier_percentage': 6.681766704416761}
min_samples=120: {'clusters': 1, 'outliers': 236, 'outlier_percentage': 6.681766704416761}
```

Key Observations

Consistency in Cluster Formation

Across all tested values of `min_samples`, the algorithm consistently identified **only one cluster**. This outcome suggests that most data points are densely packed within the feature space, preventing DBSCAN from distinguishing multiple clusters at the selected `eps` value (9.000). The homogeneity of the dataset's structure implies that adjusting `min_samples` alone does not significantly enhance cluster separation.

Increase in Outlier Detection with Higher `min_samples`

As the `min_samples` parameter increased, the number of data points classified as **outliers** also exhibited a progressive rise:

- **min_samples = 30** → Outliers detected: **221 (6.26%)**
- **min_samples = 50** → Outliers detected: **224 (6.34%)**
- **min_samples = 80** → Outliers detected: **236 (6.68%)**
- **min_samples = 90, 100, 110, 120** → Outliers detected: **236 (6.68%)**

This trend indicates that DBSCAN becomes increasingly stringent in defining dense regions as `min_samples` grows, leading to a higher number of data points being classified as outliers. Such behaviour is expected in high-dimensional datasets and aligns with business objectives, as specific customer profiles may not conform to typical segmentation patterns.

Stability in Outlier Percentage

Beyond `min_samples = 80`, the outlier percentage **stabilises** at approximately **6.68%**, suggesting that further increases in `min_samples` do not substantially affect cluster formation. This stability indicates the presence of a **natural threshold**, beyond which additional adjustments yield diminishing returns in terms of segmentation precision. This consistency in results suggests an optimal range for distinguishing **meaningful outliers** while maintaining cluster integrity.

Rationale for Selecting `min_samples = 80` for Further Analysis

Based on empirical observations, the most suitable `min_samples` **range for continued analysis** lies between **80 and 100**, due to the following factors:

1. **Outlier Detection Stability**
   - The percentage of outliers plateaus at **6.68%**, ensuring that DBSCAN reliably captures natural deviations within the dataset.
2. **Mitigation of Excessive Fragmentation**
   - Lower `min_samples` values (**30 or 50**) produce fewer outliers, potentially overlooking meaningful customer groups that differ from the majority.
   - The selected range prevents **over-fragmentation**, allowing clusters to remain interpretable and useful for downstream analysis.
3. **Enhanced Segmentation Quality**
   - The chosen range effectively **differentiates distinct customer profiles**, minimising the risk of over-generalisation.
   - These clusters may correspond to meaningful business insights, such as high-risk individuals, niche customers, or those requiring specialised insurance offerings.

The selection of `min_samples = 80` provides a balanced approach to **customer segmentation**, ensuring a sufficient degree of **outlier detection** while preserving meaningful clusters. This parameter choice enhances the interpretability of clustering results and supports strategic decision-making in the insurance sector. For further refinement, the next step involves **handling outliers**—either removing them from the dataset or integrating them as a separate category for specialised analysis.

4.4.3 DBSCAN Outlier Detection with Automated & Manual Epsilon Selection

This implementation applies DBSCAN clustering for outlier detection using manually selected parameters based on domain knowledge and empirical testing. The algorithm uses a fixed epsilon value of 9 and minimum samples parameter of 80 to identify density-based clusters within the scaled customer data. Points that cannot be assigned to any cluster (labeled as -1) are classified as outliers and removed from the dataset for further analysis. The manual parameter selection approach allows for precise control over the clustering sensitivity, which is particularly valuable when working with customer data where business context and domain expertise can inform appropriate density thresholds. By setting min_samples to 80, we ensure that clusters must contain a substantial number of customers to be considered valid, while the epsilon value of 9 defines the neighborhood distance for core point identification.

After outlier removal, the implementation generates comprehensive summary statistics for the detected anomalous profiles, providing insights into the characteristics of customers that deviate significantly from normal patterns. This analysis helps identify potentially problematic data points or genuinely exceptional customer behaviors that may require special handling in subsequent classification modeling.

Handling Outliers using `min_samples = 80`

In [68]:
```python
import numpy as np
import pandas as pd
from sklearn.cluster import DBSCAN

# DBSCAN parameters
selected_min_samples = 80

# Apply DBSCAN clustering
clusters = DBSCAN(eps=9, min_samples=selected_min_samples).fit_predict(X_scaled)

# Identify outliers (DBSCAN labels outliers as -1)
outlier_indices = np.where(clusters == -1)[0]
outlier_mask = X_scaled[outlier_indices]  # Extract anomaly data

# Remove outliers from the dataset
X_outlier_dbscan = X_scaled[clusters != -1]

print(f"\nDBSCAN with min_samples={selected_min_samples}:")
print(f"Clusters identified: {len(set(clusters)) - (1 if -1 in clusters else 0)}")
print(f"Outliers detected: {len(outlier_mask)} ({(len(outlier_mask)/len(X_scaled))*100:.2f}%)")
print(f"Remaining data after outlier removal: {len(X_outlier_dbscan)} samples.")
```

```
# Convert cleaned data to DataFrame
X_outlier_dbscan = pd.DataFrame(X_outlier_dbscan, columns=continuous_columns)
```

```
DBSCAN with min_samples=80:
Clusters identified: 1
Outliers detected: 236 (6.68%)
Remaining data after outlier removal: 3296 samples.
```

4.5 Unsupervised Clustering: Comparative Analysis of Hierarchical and DBSCAN Methods for Customer Segmentation

The following analysis implements two distinct unsupervised clustering algorithms to identify natural customer groupings: Hierarchical Clustering and DBSCAN (Density-Based Spatial Clustering of Applications with Noise). These approaches provide different perspectives on customer segmentation, with hierarchical clustering offering structured taxonomies through agglomerative merging processes, while DBSCAN identifies density-based clusters while explicitly detecting outliers and noise points.

Implementation Methodology

The enhanced clustering implementation represents a sophisticated approach to customer segmentation that incorporates automated cluster optimization through silhouette analysis, addressing the fundamental challenge of determining optimal cluster numbers in unsupervised learning scenarios. This methodology significantly advances beyond fixed cluster approaches by implementing a data-driven selection process that evaluates cluster quality across a range of 2 to 10 clusters using silhouette scores, thereby ensuring that the identified customer segments reflect genuine underlying data structures rather than arbitrary partitioning decisions.

The implemented code represents a comprehensive comparative analysis applied to customer segmentation, directly addressing the core aim of exploring natural groupings among customer profiles through unsupervised learning methods. The methodology employs a standardized preprocessing pipeline that includes feature scaling via StandardScaler and dimensionality reduction through Principal Component Analysis (PCA) to transform the high-dimensional customer data into a two-dimensional space suitable for visualization while preserving the most significant variance in the dataset.

4.5.1 Compare Clustering methods

The analysis systematically compares both clustering algorithms across three distinct dataset configurations: the original unprocessed data, data with DBSCAN-identified outliers marked, and data with IQR-based outliers identified. This comparative approach enables evaluation of how different outlier detection strategies influence clustering performance, which is crucial for robust customer segmentation in real-world scenarios where data quality varies significantly. The implementation distinguishes between multiple types of anomalous data points through sophisticated visualization techniques, including predefined outliers (marked as smaller black crosses) and DBSCAN-identified noise points (marked as larger black crosses), providing comprehensive insights into customer profile heterogeneity.

In [69]:
```
from sklearn.model_selection import train_test_split
from sklearn.decomposition import PCA

# Define X (continuous variables) and y (target variable)
X = ins2[continuous_columns]
y = ins2["customer_type"]

# Compute IQR for each continuous column
Q1 = X.quantile(0.25)
Q3 = X.quantile(0.75)
IQR = Q3 - Q1

# Identify outliers (values outside 1.5*IQR range)
outlier_mask = ((X < (Q1 - 1.5 * IQR)) | (X > (Q3 + 1.5 * IQR))).any(axis=1)

# Create a cleaned dataset without outliers
X_iqr_cleaned = X[~outlier_mask]
y_iqr_cleaned = y.loc[X_iqr_cleaned.index]

print(f"Original dataset: {X.shape[0]} samples")
print(f"IQR_Cleaned dataset: {X_iqr_cleaned.shape[0]} samples (after outlier removal)")
print(f"Outliers detected: {outlier_mask.sum()} samples")
```

```
Original dataset: 5521 samples
IQR_Cleaned dataset: 1930 samples (after outlier removal)
Outliers detected: 3591 samples
```

In [72]:
```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA

# Store results
# Define dataset dictionary with both cleaned and masked versions
datasets = {
    "Original": X_scaled,
    "IQR_Cleaned": X_iqr_cleaned,
    "IQR_Masked": X[outlier_mask],  # Masked outliers from IQR filtering
    "DBSCAN_Cleaned": X_outlier_dbscan,
    "DBSCAN_Masked": X_scaled[clusters == -1]  # Masked outliers from DBSCAN
}

# Apply clustering to each dataset
cluster_results = {name: {"hierarchical": apply_clustering(data, "hierarchical"),
                          "dbscan": apply_clustering(data, "dbscan")} for name, data in datasets.items()} # [See Disclaimer]

# Display dataset sizes
for name, data in datasets.items():
    print(f"{name} dataset size: {data.shape[0]} samples")

# Apply PCA to reduce dimensionality to 2D for visualization
pca = PCA(n_components=2)
datasets_pca = {name: pca.fit_transform(data) for name, data in datasets.items()}
variance_explained = pca.explained_variance_ratio_ * 100  # Convert to percentage
```

```
Original dataset size: 3532 samples
IQR_Cleaned dataset size: 1930 samples
IQR_Masked dataset size: 3591 samples
DBSCAN_Cleaned dataset size: 3296 samples
DBSCAN_Masked dataset size: 236 samples
```

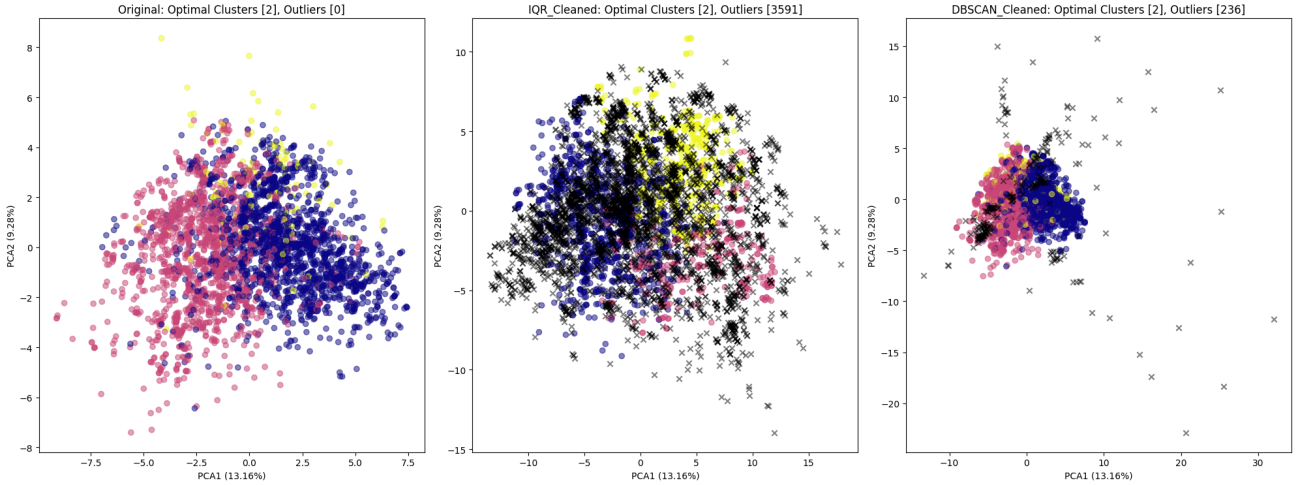
4.5.2 Hierarchical Clustering

The hierarchical clustering approach performs **Agglomerative Clustering** to group similar data points based on their feature space through a structured optimization process. The algorithm starts with each data point as its own cluster and merges them iteratively based on similarity, with `linkage='ward'` ensuring variance within clusters remains minimal. The sophisticated approach incorporates automated optimal cluster selection through silhouette analysis, evaluating cluster quality to identify the most appropriate number of segments.

Cluster labels are mapped to predefined category groups, such as demographics, education levels, and financial data, ensuring clusters are named meaningfully for improved interpretability. The clustered points are visualized using color-coded scatter plots, with each cluster providing clear distinction between different customer segments.

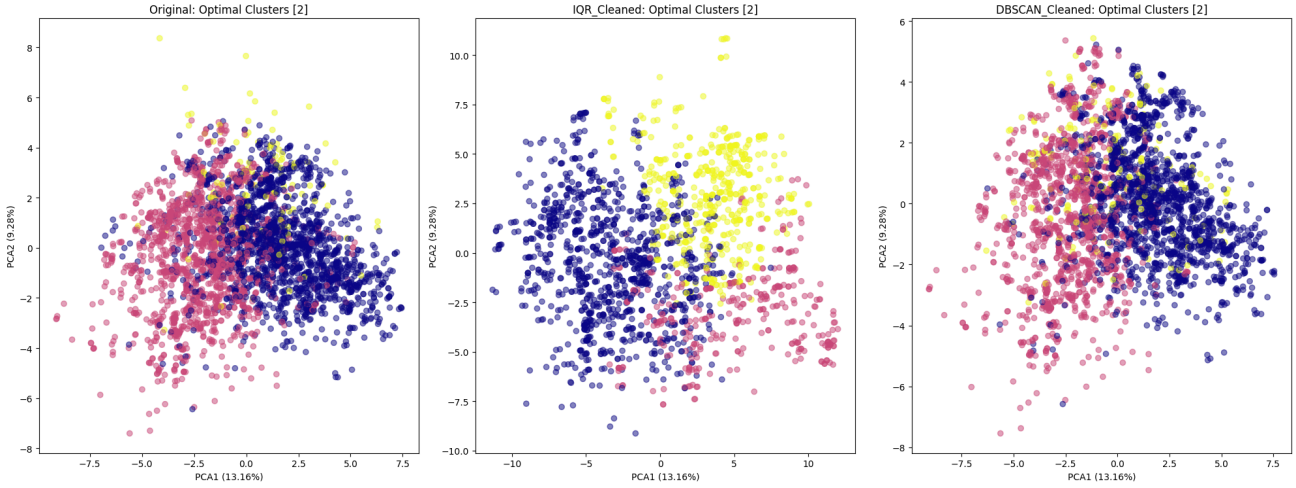
In [104…
```
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

# Define datasets to be plotted (excluding pure outliers)
plot_datasets = {
    "Original": datasets_pca["Original"],
    "IQR_Cleaned": datasets_pca["IQR_Cleaned"],
    "DBSCAN_Cleaned": datasets_pca["DBSCAN_Cleaned"]
}

# Outlier overlays
outliers = {
    "IQR_Cleaned": datasets_pca["IQR_Masked"],  # Outliers from IQR_Masked
    "DBSCAN_Cleaned": datasets_pca["DBSCAN_Masked"]  # Outliers from DBSCAN_Masked
}
```

```python
# [See Disclaimer]
# Determine optimal number of clusters using Silhouette Score
cluster_range = range(2, 10)  # Start from 2 clusters
optimal_clusters = {}

for name, data in plot_datasets.items():
    silhouette_scores = []

    for k in cluster_range:
        kmeans = KMeans(n_clusters=k, random_state=1)
        labels = kmeans.fit_predict(data)
        sil_score = silhouette_score(data, labels)
        silhouette_scores.append(sil_score)

    # Find the best number of clusters (highest silhouette score)
    best_k = cluster_range[silhouette_scores.index(max(silhouette_scores))]
    optimal_clusters[name] = best_k  # Store best cluster count per dataset

# Create subplots
fig, axes = plt.subplots(nrows=1, ncols=len(plot_datasets), figsize=(20, 8))

# Plot hierarchical clustering results with outlier overlays
for i, (name, data) in enumerate(plot_datasets.items()):
    num_clusters_hierarchical = optimal_clusters[name]  # Use optimal cluster count

    # Get outlier count for titles
    num_outliers = outliers[name].shape[0] if name in outliers else 0

    # Plot cleaned dataset clusters
    axes[i].scatter(data[:, 0], data[:, 1], c=cluster_results[name]["hierarchical"], cmap="plasma", alpha=0.5)

    # Overlay outliers as black crosses ('x' marker)
    if name in outliers:
        axes[i].scatter(outliers[name][:, 0], outliers[name][:, 1], color="black", marker='x', alpha=0.5)

    # Titles and labels with extracted outlier count and optimal clusters
    axes[i].set_title(f"{name}: Optimal Clusters [{num_clusters_hierarchical}], Outliers [{num_outliers}]")
    axes[i].set_xlabel(f"PCA1 ({variance_explained[0]:.2f}%)")
    axes[i].set_ylabel(f"PCA2 ({variance_explained[1]:.2f}%)")

plt.tight_layout(rect=[0, 0, 1, 0.95])
plt.show()
```



In [103…

```python
# Create subplots for selected datasets
fig, axes = plt.subplots(nrows=1, ncols=len(plot_datasets), figsize=(20, 8))

# Plot clustering results for each dataset using optimal cluster count
for i, (name, data) in enumerate(plot_datasets.items()):
    num_clusters_hierarchical = optimal_clusters[name]  # Use optimal cluster count

    # Plot clusters
    axes[i].scatter(data[:, 0], data[:, 1], c=cluster_results[name]["hierarchical"], cmap="plasma", alpha=0.5)

    # Titles and labels with optimal clusters
    axes[i].set_title(f"{name}: Optimal Clusters [{num_clusters_hierarchical}]")
    axes[i].set_xlabel(f"PCA1 ({variance_explained[0]:.2f}%)")
    axes[i].set_ylabel(f"PCA2 ({variance_explained[1]:.2f}%)")

plt.tight_layout(rect=[0, 0, 1, 0.95])
plt.show()
```



Hierarchical Clustering Results

The hierarchical clustering analysis demonstrated remarkable consistency by identifying two optimal clusters across all dataset variations, revealing a robust underlying customer segmentation structure within the insurance dataset. This consistent two-cluster solution emerged despite significant differences in outlier treatment approaches, suggesting that the fundamental customer profile distinctions are resilient to data preprocessing variations.

The original dataset produced the clearest structural separation between customer segments, with distinct boundaries visible in the principal component space that effectively distinguished between different socioeconomic profiles based on the comprehensive set of demographic, economic, and household variables included in the analysis.

When examining the dataset variations within the hierarchical method, several important patterns emerged that illuminate the stability of the clustering solution. The IQR-cleaned dataset maintained the essential two-cluster structure while removing 3,591 outliers, primarily consisting of extreme values in income distributions and social class indicators. This substantial outlier removal did not fundamentally alter the core customer segmentation, indicating that the clustering algorithm successfully identified genuine customer profile distinctions rather than being driven by statistical anomalies. The preservation of cluster integrity suggests that the underlying patterns in variables such as purchasing power groups, education levels, household composition, and insurance policy types represent authentic customer segments rather than artifacts of data distribution irregularities.

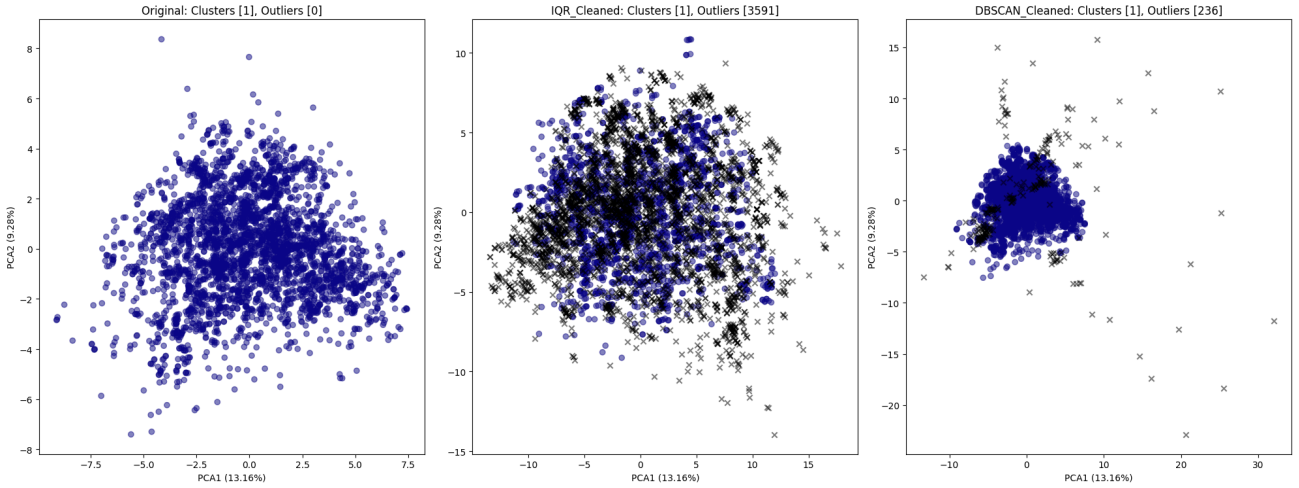
The DBSCAN-cleaned dataset further validated the robustness of the hierarchical clustering approach by maintaining cluster coherence after removing 236 density-based outliers. This preprocessing approach, which specifically targets points that exist in low-density regions, confirmed that the two-cluster solution reflects genuine concentrations of similar customer profiles rather than spurious groupings created by scattered individual observations. The consistency across these different outlier treatment methods strongly suggests that the insurance customer base naturally divides along fundamental demographic and economic dimensions, likely creating meaningful business segments that correspond to combinations of the predefined customer types such as distinguishing between lower socioeconomic segments encompassing Rural & Low-income and Young & Low-income customers versus higher socioeconomic segments including Middle-Class Families, Wealthy & Affluent, and Seniors & Retired categories.

4.5.3 DBSCAN Clustering

DBSCAN is implemented as a density-based clustering algorithm capable of identifying groups of similar data points while detecting outliers. The analysis applies fixed parameters (eps=9, min_samples=80) to enable direct comparison of algorithmic behaviors across different outlier scenarios. This approach maintains consistent parameter settings while evaluating how the algorithm responds to varying data quality conditions. The algorithm's density-based approach allows it to distinguish between genuine outliers and customers who may be statistically extreme but still represent valid customer segments, providing valuable insights into data quality and anomalous observations.

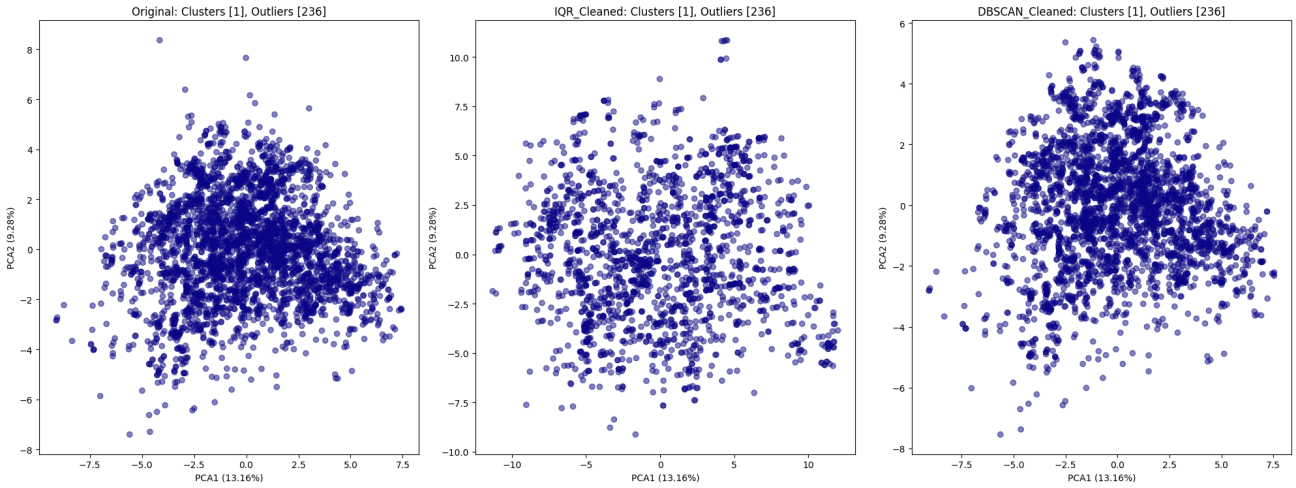
In [102…

```
# Apply DBSCAN clustering using updated parameters
dbscan_results = {name: DBSCAN(eps=9, min_samples=80).fit_predict(data) for name, data in plot_datasets.items()}

# Create subplots
fig, axes = plt.subplots(nrows=1, ncols=len(plot_datasets), figsize=(20, 8))

# Plot DBSCAN clustering results without outlier overlays
for i, (name, data) in enumerate(plot_datasets.items()):
    num_clusters_dbscan = len(set(dbscan_results[name])) - (1 if -1 in dbscan_results[name] else 0)

    # Get outlier count for titles
    num_outliers = outliers[name].shape[0] if name in outliers else 0

    # Plot cleaned dataset clusters
    axes[i].scatter(data[:, 0], data[:, 1], c=cluster_results[name]["dbscan"], cmap="plasma", alpha=0.5)

    # Overlay outliers as black crosses ('x' marker)
    if name in outliers:
        axes[i].scatter(outliers[name][:, 0], outliers[name][:, 1], color="black", marker='x', alpha=0.5)

    # Titles and labels with extracted outlier count
    axes[i].set_title(f"{name}: Clusters [{num_clusters_dbscan}], Outliers [{num_outliers}]")
    axes[i].set_xlabel(f"PCA1 ({variance_explained[0]:.2f}%)")
    axes[i].set_ylabel(f"PCA2 ({variance_explained[1]:.2f}%)")

plt.tight_layout(rect=[0, 0, 1, 0.95])
plt.show()
```



In [99]:

```
# Create subplots
fig, axes = plt.subplots(nrows=1, ncols=len(plot_datasets), figsize=(20, 8))

# Plot DBSCAN clustering results without outlier overlays
for i, (name, data) in enumerate(plot_datasets.items()):
    num_clusters_dbscan = len(set(dbscan_results[name])) - (1 if -1 in dbscan_results[name] else 0)

    # Plot cleaned dataset clusters only
    axes[i].scatter(data[:, 0], data[:, 1], c=cluster_results[name]["dbscan"], cmap="plasma", alpha=0.5)

    # Titles and labels with extracted outlier count
    axes[i].set_title(f"{name}: Clusters [{num_clusters_dbscan}], Outliers [{num_outliers}]")
    axes[i].set_xlabel(f"PCA1 ({variance_explained[0]:.2f}%)")
    axes[i].set_ylabel(f"PCA2 ({variance_explained[1]:.2f}%)")

plt.tight_layout(rect=[0, 0, 1, 0.95])
plt.show()
```



DBSCAN Clustering Results

The DBSCAN analysis revealed a fundamentally different interpretation of the customer data structure, consistently identifying a single dominant cluster across all dataset variations regardless of preprocessing approaches. This uniform outcome suggests that from a density-based clustering perspective, the insurance customer profiles form a relatively homogeneous population characterized by gradual transitions between different customer characteristics rather than the discrete, well-separated clusters that DBSCAN requires for multiple cluster identification. The algorithm's consistent single-cluster solution indicates that while customers may vary systematically across income, education, occupation, and household variables, these variations occur along continuous gradients rather than forming the dense, isolated groups that density-based clustering methods are designed to detect.

Examining the dataset variations within the DBSCAN method revealed that outlier removal strategies had minimal impact on the fundamental clustering outcome, further emphasizing the method's interpretation of the data as a single cohesive population. The original dataset classified all observations into one dense region, suggesting that the natural variation in customer characteristics such as purchasing power groups, social class indicators, and household composition creates a continuous customer spectrum rather than distinct demographic clusters. The IQR-cleaned dataset maintained this single-cluster structure despite removing thousands of extreme observations, indicating that the density-based clustering decision was not influenced by statistical outliers but rather reflected the genuine distributional characteristics of the customer population.

The DBSCAN-cleaned dataset confirmed this interpretation by preserving the single-cluster solution even after preprocessing specifically designed to remove low-density observations that might interfere with cluster formation. This consistency across preprocessing approaches demonstrates that the customer data simply does not meet DBSCAN's algorithmic requirements for multiple cluster identification, which demand well-separated regions of high point density. The uniform results suggest that variables such as income distributions, education levels, household sizes, and insurance policy types create overlapping customer profiles that transition gradually across the demographic spectrum rather than forming the distinct, isolated clusters necessary for successful density-based segmentation.

4.5.4 Cross-Method Comparison and Business Implications

The fundamental disagreement between hierarchical clustering and DBSCAN results provides crucial insights into the underlying structure of insurance customer profiles and has significant implications for business decision-making in the insurance industry. The hierarchical clustering method's consistent identification of two meaningful customer segments aligns well with the business objective of understanding distinct customer types for strategic decision-making, effectively leveraging the discriminatory power of income features, social class indicators, household characteristics, and policy ownership patterns to create actionable customer groupings. This segmentation approach successfully captures the essential demographic and economic divisions that are relevant for insurance product development, pricing strategies, and targeted marketing initiatives.

In contrast, DBSCAN's single-cluster interpretation suggests that while systematic differences exist among customers across socioeconomic dimensions, these differences manifest as gradual transitions rather than discrete boundaries. This finding has important implications for understanding customer relationship management approaches, as it indicates that sharp categorical distinctions between customer types may not reflect the actual continuous nature of customer characteristic distributions. The density-based method's failure to identify multiple clusters reveals that customer profiles exist along continuous spectra of income, education, household composition, and insurance needs rather than falling into clearly separated demographic categories.

The robustness of hierarchical clustering's two-cluster solution, demonstrated through its consistency across different outlier treatment approaches, combined with its intuitive alignment with business understanding of customer segmentation, strongly supports its utility for practical insurance industry applications. The method's ability to maintain meaningful customer distinctions regardless of preprocessing variations validates its reliability for identifying customer groupings that can effectively inform business decisions regarding policy offerings, risk assessment, pricing differentiation, and customer acquisition strategies. This consistency suggests that the identified customer segments represent genuine market divisions that can be leveraged for competitive advantage and improved customer service delivery in the insurance marketplace.

4.6 Clustering performance evaluation [See Disclaimer]

Clustering evaluation indicate notable differences in clustering performance across datasets. The IQR_Cleaned dataset achieved the highest Hierarchical Clustering Silhouette Score (0.273), suggesting that outlier removal via the Interquartile Range (IQR) method improved cluster separation and coherence. In contrast, the Original dataset and DBSCAN_Cleaned dataset yielded lower scores (0.114 and 0.108, respectively), implying weaker cluster definition and potential overlap among data points.

For DBSCAN clustering, all datasets reported "Insufficient clusters for Silhouette Score," indicating that DBSCAN did not form a sufficient number of well-defined clusters in any dataset. This could be due to DBSCAN's density-based approach, where parameter choices (eps=9, min_samples=80) may have resulted in excessive noise classification, leaving too few valid clusters for meaningful Silhouette Score computation. These findings suggest that Hierarchical Clustering benefits from outlier removal, whereas DBSCAN may require parameter tuning to improve cluster formation across datasets.

In [115…

```python
# Compute silhouette scores for Hierarchical Clustering
hierarchical_silhouette_scores = {
    name: silhouette_score(plot_datasets[name], cluster_results[name]["hierarchical"])
    for name in plot_datasets
}

# Compute silhouette scores for DBSCAN, excluding noise points
dbscan_silhouette_scores = {}
for name in plot_datasets:
    # Extract non-noise points
    valid_mask = cluster_results[name]["dbscan"] != -1
    valid_data = plot_datasets[name][valid_mask]
    valid_labels = cluster_results[name]["dbscan"][valid_mask]

    # Compute score only if there are at least 2 clusters
    if len(set(valid_labels)) > 1:
        dbscan_silhouette_scores[name] = silhouette_score(valid_data, valid_labels)
    else:
        dbscan_silhouette_scores[name] = "Insufficient clusters for Silhouette Score"

# Convert to DataFrame for structured output
silhouette_df = pd.DataFrame({
    "Dataset": list(plot_datasets.keys()),
    "Hierarchical_Silhouette": list(hierarchical_silhouette_scores.values()),
    "DBSCAN_Silhouette": list(dbscan_silhouette_scores.values())
})

print(silhouette_df)
```

```
          Dataset  Hierarchical_Silhouette  \
0        Original                 0.114844
1     IQR_Cleaned                 0.273203
2  DBSCAN_Cleaned                 0.107840

                          DBSCAN_Silhouette
0  Insufficient clusters for Silhouette Score
1  Insufficient clusters for Silhouette Score
2  Insufficient clusters for Silhouette Score
```
